

Advanced Skill Composition

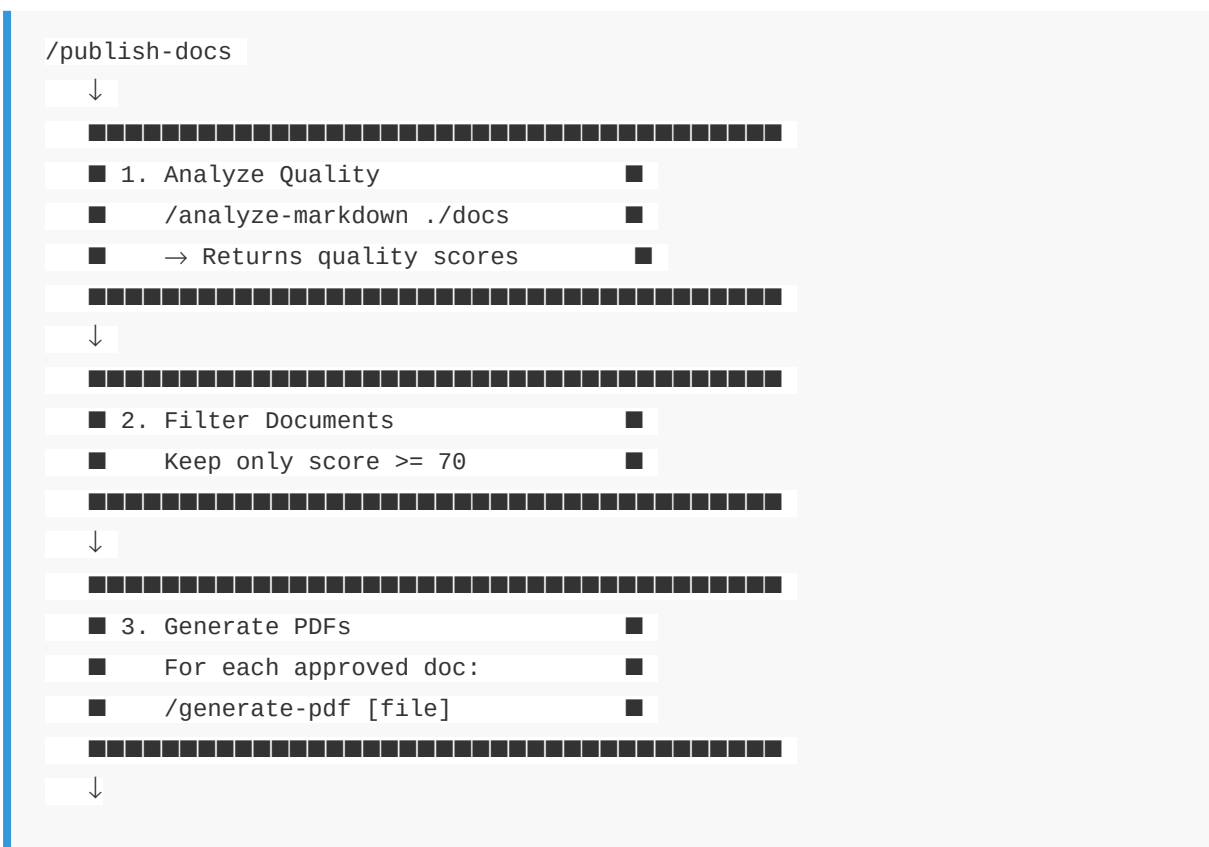
Example

■ Scenario: Document Publishing Pipeline

Goal: Create a complete documentation publishing system that: 1. Analyzes markdown quality 2. Generates PDFs only for approved documents 3. Creates an HTML index 4. Uploads to S3 5. Sends Slack notification

This demonstrates: - Multi-skill orchestration - Conditional workflows - Data passing between skills
- Error handling and rollback

■ Architecture



```
████████████████████████████████████████████████████████████████████████████████
```

```
■ 4. Create Index ■
```

```
■ /create-index ./output/pdfs ■
```

```
████████████████████████████████████████████████████████████████████████████████
```

```
↓
```

```
████████████████████████████████████████████████████████████████████████████████
```

```
■ 5. Upload to S3 ■
```

```
■ /upload-to-s3 ./output ■
```

```
████████████████████████████████████████████████████████████████████████████████
```

```
↓
```

```
████████████████████████████████████████████████████████████████████████████████
```

```
■ 6. Notify Team ■
```

```
■ /send-slack "Published X docs" ■
```

```
████████████████████████████████████████████████████████████████████████████████
```

■ ■ Implementation

Skill 1: analyze-markdown

File: `.claude/skills/analyze-markdown/SKILL.md`

```
---
```

```
name: analyze-markdown
```

```
description: Analyze markdown documents for quality and readability
```

```
disable-model-invocation: true
```

```
argument-hint: [directory]
```

```
---
```

```
■  
# Markdown Quality Analyzer
```

```
■  
Analyze all markdown files in $ARGUMENTS for quality.
```

```
■  
## Analysis Criteria
```

```
■  
For each file, score on:
```

```
- **Structure** (0-30): Proper heading hierarchy, sections
```

```
- **Readability** (0-30): Flesch-Kincaid score, sentence complexity
```

```
- **Completeness** (0-40): Has intro, examples, conclusion
```

****Total score: 0-100****

Output Format

Generate a JSON report:

```json

```
{
 "analyzed": "2026-02-24T21:00:00Z",
 "documents": [
 {
 "file": "guide.md",
 "scores": {
 "structure": 28,
 "readability": 25,
 "completeness": 35,
 "total": 88
 },
 "approved": true,
 "issues": []
 },
 {
 "file": "draft.md",
 "scores": {
 "structure": 15,
 "readability": 20,
 "completeness": 25,
 "total": 60
 },
 "approved": false,
 "issues": [
 "Missing introduction",
 "No code examples"
]
 }
],
 "summary": {
 "total": 2,
 "approved": 1,
 "needs_work": 1
 }
}
```

Save report to `output/analysis/quality-report.json`

```
Skill 2: create-index

File: `.claude/skills/create-index/SKILL.md`

```markdown
---
name: create-index
description: Generate an HTML index page for a directory of PDFs
disable-model-invocation: true
argument-hint: [pdf-directory]
---

# PDF Index Generator

Create a professional HTML index for all PDFs in $ARGUMENTS.

## Process

1. **Scan directory** for all `.pdf` files
2. **Extract metadata**:
   - File name
   - File size
   - Creation date
   - Page count (if possible)
3. **Generate HTML** with:
   - Sortable table
   - Download links
   - File size badges
   - Search filter
4. **Save** to `$ARGUMENTS/index.html`

## Styling

Use clean, professional design:
- Responsive layout
- Professional fonts
- Hover effects on links
- Mobile-friendly

## Output
```

```

```html
<!DOCTYPE html>
<html>
<head>
 <title>Documentation Index</title>
 <!-- Professional CSS -->
</head>
<body>
 <h1>Documentation Library</h1>
 <input type="search" placeholder="Filter documents...">

 <table>
 <thead>
 <tr>
 <th>Document</th>
 <th>Size</th>
 <th>Date</th>
 <th>Download</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <td>User Guide</td>
 <td>2.3 MB</td>
 <td>2026-02-24</td>
 <td>Download</td>
 </tr>
 </tbody>
 </table>
</body>
</html>

```

```

Skill 3: upload-to-s3

```

```

File: `.claude/skills/upload-to-s3/SKILL.md`

```

```

```markdown

```

```

---
```

```

name: upload-to-s3

```

```

description: Upload files to S3 bucket with public access

```

```

disable-model-invocation: true

```

```

argument-hint: [directory]

```



```

---
name: send-slack
description: Send a message to Slack channel
disable-model-invocation: true
argument-hint: [message]
---

# Slack Notifier

Send notification to team Slack channel: **$ARGUMENTS**

## Configuration

Read from environment:
- `SLACK_WEBHOOK_URL` - Webhook URL for #docs channel

## Message Format

```json
{
 "text": "$ARGUMENTS",
 "blocks": [
 {
 "type": "section",
 "text": {
 "type": "mrkdown",
 "text": "$ARGUMENTS"
 }
 }
]
}

```

## Send

```

curl -X POST $SLACK_WEBHOOK_URL \
 -H 'Content-Type: application/json' \
 -d '{"text": "$ARGUMENTS"}'

```

If successful, report confirmation.

```

Master Orchestrator: publish-docs

File: `.claude/skills/publish-docs/SKILL.md`

```markdown
---
name: publish-docs
description: Full documentation publishing pipeline with quality checks
disable-model-invocation: true
argument-hint: [docs-directory]
context: fork
agent: Explore
---

# Documentation Publishing Pipeline

Publish documentation from **$ARGUMENTS** with quality checks and deployment.

## Pipeline Steps

### 1. Analyze Quality

```

/analyze-markdown \$ARGUMENTS

```

**Decision point:** Only continue with documents scoring ≥70

### 2. Generate PDFs

For each approved document:

```

/generate-pdf [filename]

```

Track successes and failures.

### 3. Create Index

Generate browsable HTML index:

```

/create-index output/pdfs

4. Upload to S3

Deploy all generated files:

```
/upload-to-s3 output
```

Capture public URLs from output.

5. Notify Team

Send success notification with stats:

```
/send-slack "■ Published X documents to S3\n\nApproved: X/Y\nPDFs: X generated\nIndex: https://..."
```

Error Handling

- If analysis fails → Stop pipeline
- If PDF generation fails for one doc → Log and continue
- If S3 upload fails → Rollback (delete partial uploads)
- If Slack fails → Continue (notification is optional)

Summary Report

After completion, generate summary:

A progress bar with three sections: 'Documentation' (0-100%), 'Publishing' (0-100%), and 'Complete' (0-100%). The 'Documentation' section is filled with 100% black bars. The 'Publishing' section is filled with 100% black bars. The 'Complete' section is filled with 100% black bars.

■ Quality Analysis: • Total documents: 10 • Approved: 8 • Rejected: 2

■ PDF Generation: • Generated: 8 PDFs • Failed: 0

■ Deployment: • Uploaded: 9 files (8 PDFs + 1 index.html) • Base URL: <https://my-docs-bucket.s3.amazonaws.com/docs/>

■ Notification: • Slack: ■ Sent to #docs

■ Rejected Documents: • draft-incomplete.md (score: 45/100) • notes-temp.md (score: 30/100)

■ Next Steps: 1. Review rejected documents 2. Improve and re-run: `/publish-docs ./docs` 3. Access published docs: [Base URL]

Configuration Required

Before running, ensure environment variables are set:

```
```bash
export S3_BUCKET="my-docs-bucket"
export S3_PREFIX="docs/"
export AWS_REGION="us-east-1"
export SLACK_WEBHOOK_URL="https://hooks.slack.com/services/..."
```

Or create `.env` file in workspace.

---

### ## ■ Usage

#### ### Basic Usage

```
```bash
# Publish all docs with full pipeline
/publish-docs ./documentation
```

Step-by-Step (For Testing)

```
# 1. Analyze first
/analyze-markdown ./documentation

# 2. Review quality report
# (Check output/analysis/quality-report.json)

# 3. Generate PDFs for approved docs
/markdown-to-pdf ./documentation

# 4. Create index
/create-index output/pdfs

# 5. Upload
/upload-to-s3 output

# 6. Notify
```

```
/send-slack "Published documentation manually"
```

■ Data Flow

```
docs/
■■■ guide.md ■■■■■■■■■■
■■■ tutorial.md ■■■■■■■■■■> analyze-markdown
■■■ draft.md ■■■■■■■■■■ ↓
                                quality-report.json
                                ↓
                                Filter (score >= 70)
                                ↓
                                guide.md, tutorial.md
                                ↓
                                generate-pdf (each)
                                ↓
                                output/pdfs/
                                ■■■ guide.pdf
                                ■■■ tutorial.pdf
                                ↓
                                create-index
                                ↓
                                output/pdfs/index.html
                                ↓
                                upload-to-s3
                                ↓
                                S3 URLs saved
                                ↓
                                send-slack
                                ↓
                                Team notified ■
```

■ Error Recovery

Scenario: S3 Upload Fails

Problem: Network issue during upload

Recovery: 1. Pipeline detects S3 error 2. Lists partially uploaded files 3. Deletes partial uploads (cleanup) 4. Reports error with instructions 5. User can retry: `/upload-to-s3 output`

Scenario: One PDF Fails

Problem: Malformed markdown in one file

Recovery: 1. PDF generation fails for `broken.md` 2. Pipeline logs error 3. Continues with remaining files 4. Reports summary showing failure 5. User can fix `broken.md` and re-run just that file: `/generate-pdf docs/broken.md`

■ Benefits of This Architecture

1. **Modular** - Each skill is independent, reusable
2. **Testable** - Can test each skill individually
3. **Flexible** - Can run full pipeline or individual steps
4. **Resilient** - Continues on non-critical failures
5. **Observable** - Clear reporting at each step
6. **Maintainable** - Update one skill without touching others

■ Key Takeaways

Orchestration Pattern

Master orchestrator (`publish-docs`) calls worker skills in sequence: - Each worker does ONE job well - Master handles errors and retry logic - Workers report status clearly - Master generates comprehensive summary

Data Passing

- **Quality scores** → determine which docs to publish
- **PDF paths** → input to index generator
- **S3 URLs** → included in Slack notification
- **Error messages** → aggregated in final report

Error Handling

- **Critical errors** (analysis fails) → Stop pipeline
- **Per-item errors** (one PDF fails) → Log and continue
- **Optional steps** (Slack) → Skip if fails, don't block
- **Rollback** (S3 partial) → Clean up on failure

Advanced example demonstrating multi-skill orchestration - 2026-02-24